

Memory System CPI/Timing Data

The table below summarizes the CPI improvements and icetime reported frequency for different configurations of caches.

Summary of Results for I-Cache and D-Mem Lookahead Improvements						
ENABLE_IRQ = 0						
Configuration	Baseline	Regular I-Cache icache_multiword	I-Cache with Lookahead icache_multiword_lookahead	I-Cache with Lookahead + data memory LA buffer	I-Cache with Lookahead + data memory LA buffer	I-Cache with Lookahead + data memory LA buffer
Cache Size	-	128x16	128x16	128x16	256x8	64 sets x 2 ways x 16 words
ICESTORM_LC (out of 5280)	4470	4550	4660	4767	5056	5253
ICESTORM_RAM (out of 30)	4	22	22	22	22	24
Achieved Fmax (MHz)	19.71	19.07	18.50	18.89	17.88	18.59
Bubble Sort Benchmark						
Achieved CPI	10.9106	5.5699	4.5159	4.1488	4.1490	4.1488
Time per instruction (ns)	553.56	292.08	244.10	219.63	232.05	223.17
Basic Integral Benchmark						
Achieved CPI	9.9373	5.9998	5.0736	4.7034	4.7034	4.7034
Time per instruction (ns)	504.18	314.62	274.25	248.99	263.05	253.01
Final Benchmark						
		This benchmark was designed specifically to punish non-associative caches.				
Achieved CPI	10.1860	10.7539	9.8009	9.5407	9.0461	4.7135
Time per instruction (ns)	516.79	563.92	529.78	505.07	505.93	253.55
Average Results						
Achieved CPI	10.34463	7.4412	6.4635	6.1310	5.9662	4.5219
Time per instruction (ns)	524.84	390.20	349.38	324.56	333.68	243.24
Total Speed Multiplier	1x	1.345x	1.502x	1.617x	1.573x	2.158x
Results are averaged over three real-world tasks. The first two benchmarks have a small loop of around ~30 instructions, while the third benchmark has a much larger loop of 1700 instructions.						
The final benchmark also is designed such that memory accesses may cause conflict misses in a non-associative cache, even before cache capacity is reached, so CPI is high for all but the last configuration.						
Max size cache (by LUT & BRAM availability, preferring lines over words per line), is used for all configurations. It was found that using a smaller cache did not result in noticeable Fmax improvement.						
Data memory lookahead buffer offers consistent gains of 0.25-0.4 CPI, completely separate from cache hit/miss behaviour (it depends on the number of load instructions).						
If QDDR mode SPI is not enabled, CPI on cache miss penalty skyrockets to around 60.						

The data is collected on 3 different benchmarks: a simple bubble sort benchmark (provided), a simple basic polynomial integral benchmark, and our final benchmark: the multiple-polynomial integral with sine LUT benchmark designed to specifically punish smaller, non-associative caches (see the Benchmark Design document).

Note that icetime timing reported value is included for indicative purposes only to allow a quick comparison. Actual achievable frequencies exceed the timing report values which are conservative and for worst-case hardware.

Overall, we achieve an indicative total speed multiplier of 2.158x on this set of three benchmarks

All the caches here are based on the multiword cache with 1-cycle BRAM access, which was then iteratively built upon.

The first improvement was to use the CPU mem_la_* interface to lookahead at upcoming instruction fetches. This eliminated the 1-cycle hit/miss check and allows zero-cycle return of the cache hit data by the time the instruction fetch request arrives at the standard interface.

Next, a data memory lookahead buffer was added, also using the lookahead interface, so data memory reads (SPRAM) take 0 cycles instead of 1. This reduces 1 CPI for each load instruction, which is often around 20-37% of the total number of instructions.

Finally, the cache was made to be associative, with two "ways". See the benchmark design section above for data that highlights how this improves CPI.

From the data we can see that each of these changes improved performance in the three different workloads.

Note that only data for the largest caches have been presented here. It was found that smaller caches don't really have that much timing impact so we probably shouldn't need to use smaller caches.

Note we also tried 256x8 instead of 128x16: this was pretty detrimental to timing though. And with the timing-efficient versions of the cache (with lookahead) and dmem lookahead, we can't fit 256x8 although it would be nice.